

# Question Bank — Week 3

Recursion · data structures and classes in Python (Lecture 03, Lab 03) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

**Questions:** *week03-questions.md*. Commit to your own answer before reading one here.

## Part A — Multiple choice

| Q   | Ans | Q   | Ans | Q   | Ans | Q   | Ans |
|-----|-----|-----|-----|-----|-----|-----|-----|
| M01 | a   | M08 | c   | M15 | d   | M22 | d   |
| M02 | d   | M09 | b   | M16 | c   | M23 | b   |
| M03 | b   | M10 | a   | M17 | a   | M24 | c   |
| M04 | c   | M11 | d   | M18 | b   | M25 | b   |
| M05 | a   | M12 | c   | M19 | d   | M26 | d   |
| M06 | d   | M13 | b   | M20 | c   |     |     |
| M07 | b   | M14 | a   | M21 | a   |     |     |

**W3-M01 — a.** A base case that answers directly, and progress towards it. A loop (b) is not needed; one recursive call (c) is enough; a global counter (d) is a debugging aid, not a requirement.

**W3-M02 — d.**  $-1, -2, -3, \dots$  moves *away* from the base case 0, so the calls never stop until Python's depth limit. `ValueError` (c) is what the exercise version should raise — by checking  $n < 0$  explicitly.

**W3-M03 — b.** `sys.getrecursionlimit()` is 1,000 by default.

**W3-M04 — c.**  $n + 1$  frames are on the stack at the deepest point ( $n$  down to 0). Space is decided by the depth, not by the answer's size (d).

**W3-M05 — a.** Unroll:  $c$  is added once per halving,  $\log_2 n$  times.

**W3-M06 — d.** The number of calls doubles at every level:  $1 + 2 + 4 + \dots + 2^{n-1}$ .

**W3-M07 — b.**  $cn + c(n-1) + \dots + c = cn(n+1)/2$ .

**W3-M08 — c.**  $2^5 - 1 = 31$ .

**W3-M09 — b.** Each item is either in or out:  $2^4 = 16$ . (24 (c) is  $4!$ , the number of permutations.)

**W3-M10 — a.**  $4! = 4 \times 3 \times 2 \times 1 = 24$ .

**W3-M11 — d.** `fib(5)` calls `fib(4)` and `fib(3)`, and so on: the recursion tree has 15 nodes. (In general  $2 \text{fib}(n+1) - 1$ .) 8 (b) is the *value* `fib(6)`.

**W3-M12 — c.** Two calls on  $n/2$ :  $T(n) = 2T(n/2) + c = O(n)$ . Compute the half power **once**, store it, and square the variable:  $O(\log n)$ .

**W3-M13 — b.** Each call copies the rest of the list:  $(n-1) + (n-2) + \dots = O(n^2)$  — the hidden loop from Lecture 02.

**W3-M14 — a.** Solve each subproblem once, store its answer (for Fibonacci, in an Array indexed by  $n$ ), and look it up afterwards.

**W3-M15 — d.** Frames are popped as calls return; the deepest chain is `hanoi(20) → hanoi(19) → ... → hanoi(1)`: about 20 frames. The *total* number of calls is exponential; the *depth* is linear.

**W3-M16 — c.** Each call prints  $n$  on the way **down**, then again on the way **back up**, after the inner call has returned: 3 2 1 then 1 2 3.

**W3-M17 — a.** The empty set has exactly one subset — itself. Returning `[]` (b) means “no subsets”, and every result built from it is then empty too.

**W3-M18 — b.** Python deliberately keeps every frame (it keeps `tracebacks` complete). Deep recursion should become a loop.

**W3-M19 — d.** `sort()` sorts in place and returns `None`. `sorted(marks)` returns the new list.

**W3-M20 — c.** `{}` is an empty dictionary; an empty set is `set()`.

**W3-M21 — a.** A set is a hash table: average  $O(1)$  membership. A list would be  $O(n)$ .

**W3-M22 — d.** `tricks` is a **class** variable: one list shared by every instance. Create per-object data in `__init__`: `self.tricks = []`.

**W3-M23 — b.** The body only starts running when `next()` is called — by a `for` loop, `list()` and so on.

**W3-M24 — c.** Every remaining item shifts one place left. Use `collections.deque` in plain Python — or, in this course, your own `CircularQueue` (Week 7).

**W3-M25 — b.** `get(w, 0)` supplies 0 for a missing key, so no `KeyError`.

**W3-M26 — d.** Each call spawns two subtrees almost as large as itself; the call count grows by a factor of about the golden ratio, 1.618, per step of  $n$ .

---

## Part B — Short answer and essay

**W3-E1 (4)** — writing `f` for `factorial`, the stack at each step, bottom → top (Lecture 03’s figure draws the same thing for `factorial(4)`):

| Step | Stack             | What happens            |
|------|-------------------|-------------------------|
| 1    | <code>f(3)</code> | calls <code>f(2)</code> |

| Step | Stack               | What happens                      |
|------|---------------------|-----------------------------------|
| 2    | f(3) f(2)           | calls f(1)                        |
| 3    | f(3) f(2) f(1)      | calls f(0)                        |
| 4    | f(3) f(2) f(1) f(0) | <b>base case:</b> f(0) returns 1  |
| 5    | f(3) f(2) f(1)      | $1 \times 1 = 1$ ; f(1) returns 1 |
| 6    | f(3) f(2)           | $2 \times 1 = 2$ ; f(2) returns 2 |
| 7    | f(3)                | $3 \times 2 = 6$ ; f(3) returns 6 |

Marking points: each call pushes a frame with its own  $n$ ; nothing is multiplied during winding; the base case stops the growth; each return pops a frame and the frame below finishes its multiplication; maximum depth  $4 = n + 1$ .

### W3-E2 (4)

- `fib(n)` calls `fib(n-1)` **and** `fib(n-2)`; the recursion tree roughly doubles with each increase in  $n$ , because each node has two children almost as large as itself.
- The same subproblems are recomputed many times: in `fib(5)`, `fib(2)` is computed 3 times, `fib(3)` twice. Total calls  $2 \text{fib}(n + 1) - 1$  — exponential, about  $1.618^n$ ; `fib(30)` makes 2,692,537 calls.
- **Memoisation:** store each `fib(k)` in a memo Array the first time it is computed; later calls look it up. Each value is computed once.
- New cost:  $O(n)$  time ( $2n - 1$  calls — 59 for  $n = 30$ ),  $O(n)$  space for the memo plus  $O(n)$  stack depth.

### W3-E3 (3)

- **Recursion** when the data or the problem is recursive: trees, nested lists, grammars; divide and conquer (binary search, merge sort); branching searches (subsets, Hanoi, backtracking) — where it is clearer and the depth is small.
- **A loop** when the recursion would be one call per element of flat data, or the depth could be large.
- **Python concerns**, any two: the  $\sim 1,000$ -frame recursion limit (`RecursionError`); no tail-call optimisation, so every call costs a frame; a function call is relatively slow in CPython; slicing inside recursion silently adds  $O(n)$  per call.

### W3-E4 (4)

- **Strategy** for  $n$  disks from A to C: move  $n - 1$  disks A  $\rightarrow$  B (recursively, using C as spare); move the largest disk A  $\rightarrow$  C; move the  $n - 1$  disks B  $\rightarrow$  C (recursively, using A as spare). Base case: one disk, one move.
- **Recurrence:**  $T(1) = 1, T(n) = 2T(n - 1) + 1$ .
- **Solution:**  $T(n) = 2^n - 1$  (unroll: 1, 3, 7, 15, ...; or the induction proof in W3-R5).
- **Minimum:** the largest disk must move at least once. At that moment all the other  $n - 1$  disks must be on the spare peg — which takes at least  $T(n - 1)$  moves — and afterwards they must all be moved back on top, at least  $T(n - 1)$  more. So any solution needs at least  $2T(n - 1) + 1$  moves: the exponential cost is in the problem, not the algorithm.

### W3-E5 (3)

- When writing the recursive case, **assume** the recursive call correctly solves the smaller problem, and only ask how to build this answer from that one. Do not trace every call.
- This is **induction**: the base case is the induction base; “if the smaller call is right then this call is right” is the inductive step. Together they prove the function correct for every input the measure reaches.
- It only works if the base case is right and the recursive case really makes progress towards it.

### W3-E6 (4)

- (a) **list** — ordered, grows by append in amortised  $O(1)$ , allows duplicates (two students can have the same mark).
- (b) **set** — “have I seen this ID?” is  $O(1)$  on average against  $O(n)$  for a list; duplicates are exactly what it prevents.
- (c) **dict** —  $ID \rightarrow name$ ,  $O(1)$  average lookup by key.
- (d) **tuple** — a fixed record of related values of different types; immutable, therefore hashable, so it can be a dictionary key or set member (a list cannot).

## Part C — Recurrences

**W3-R1.** One call on a number with one digit fewer,  $O(1)$  work:  $T(1) = c$ ,  $T(n) = T(n-1) + c \rightarrow T(n) = O(n)$  in the number of digits, i.e.  $O(\log x)$  in the value  $x$ .

**W3-R2.** Two calls on  $n-1$ :  $T(1) = c$ ,  $T(n) = 2T(n-1) + c \rightarrow O(2^n)$ . It returns  $2^{n-1}$  (for  $n \geq 1$ ): 1, 2, 4, 8, ...  $O(n)$  **version** — call once, double the result:

```
def mystery(n):
    if n <= 1:
        return 1
    return 2 * mystery(n - 1)
```

$O(1)$  **version** (arithmetic on small numbers counted as  $O(1)$ ): return  $2^{n-1}$  for  $n \geq 1$ , else 1. Recognising what a recursion *computes* can remove it entirely.

**W3-R3.** Two calls on halves,  $O(1)$  work:  $T(1) = c$ ,  $T(n) = 2T(n/2) + c$ . The recursion tree has  $1 + 2 + 4 + \dots + n \approx 2n$  nodes, each  $O(1)$ :  $O(n)$  — the same as a loop, not  $O(n \log n)$  (that needs  $O(n)$  work per call, as in merge sort). The stack depth is the height of the tree:  $\log_2 n$ , so  $O(\log n)$  space — much less than a one-element-at-a-time recursion, which would need depth  $n$ .

**W3-R4.** `text[1:]` copies  $n-1$  characters and the concatenation builds a string of length  $n$ :  $O(n)$  work outside one call on  $n-1$ .  $T(0) = c$ ,  $T(n) = T(n-1) + cn \rightarrow O(n^2)$ .

**W3-R5.** Claim:  $T(n) = 2^n - 1$  for all  $n \geq 1$ .

- **Base:**  $T(1) = 1 = 2^1 - 1$ , true.
- **Step:** assume  $T(k) = 2^k - 1$  for some  $k \geq 1$ . Then  $T(k+1) = 2T(k) + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$ , as required.

- By induction the claim holds for every  $n \geq 1$ . ■

## Part D — Trace the code and the call stack

### W3-T1

19 7

g adds the digits:  $4 + 0 + 9 + 6 = 19$ .  $g(7)$  is the base case.

### W3-T2

1101 1 1000

The binary representations of 13, 1 and 8. The leading digits are built by the inner calls; each call appends its own last bit on the way back.

### W3-T3

| Call                 | $a \% b$        | Returns |
|----------------------|-----------------|---------|
| $\text{gcd}(48, 18)$ | $48 \% 18 = 12$ | 6       |
| $\text{gcd}(18, 12)$ | $18 \% 12 = 6$  | 6       |
| $\text{gcd}(12, 6)$  | $12 \% 6 = 0$   | 6       |
| $\text{gcd}(6, 0)$   | base case       | 6       |

Maximum depth: **4** frames. Every frame returns the same value unchanged — a tail call, which is why gcd converts to a loop so easily.

### W3-T4

8 25

$\text{fib}(6) = 8$ ; the call count is  $2 \text{fib}(7) - 1 = 2 \times 13 - 1 = 25$ .

**W3-T5.**  $\text{hanoi}(2, "A", "C", "B")$ : **A→B, A→C, B→C**. The moves of  $\text{hanoi}(3, "A", "C", "B")$  are A→C, A→B, C→B, **A→C**, B→A, B→C, A→C: the 4th move is **A→C** — the largest disk, exactly in the middle.

### W3-T6

[2, 1, 0, 1, 2] 11

Each level adds  $n$  on both sides of the inner result: length  $2n + 1$ , so 11 for  $n = 5$ .

### W3-T7

```
{'the': 2, 'cat': 1, 'hat': 1}
the {3}
```

Dictionaries keep insertion order. Sorting the keys by negative count puts the first. The set comprehension removes duplicate lengths: all four words have 3 letters.

**W3-T8**

```
[3, 2, 1] 6 3
```

`__iter__` is a generator, and each `for/list()/sum()` calls it again, so the object can be iterated any number of times.

---

**Part E — Find and fix the bug**

**W3-B1.** For even  $n$  it makes **two** identical calls on  $n/2$ :  $T(n) = 2T(n/2) + c$ , which is  $O(n)$  — the halving saves nothing. **Fix:** call once and reuse the result:

```
def power(x, n):
    if n == 0:
        return 1
    half = power(x, n // 2)
    result = half * half
    if n % 2 == 1:
        result = result * x
    return result
```

Now  $T(n) = T(n/2) + c = O(\log n)$ .

**W3-B2.** The base case returns `[]` — “there are no subsets” — so `rest` is always empty and so is every result: `subsets([1, 2])` returns `[]`. **Fix:** return  `[[] ]` — the empty set has exactly one subset, the empty one.

**W3-B3.** Strings have `__iter__`, so “ab” is treated as a nested list; every character is itself a one-character string, which again has `__iter__`, and the function recurses into “a” for ever: `RecursionError`. **Fix:** descend only into lists: `if isinstance(item, list):`.

**W3-B4.** The recursive case computes the sum but does not **return** it, so every call except the base case returns `None`. `total([5])` gives `None`; `total([1, 2])` raises `TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'`, one level above the base case. **Fix:** `return values[i] + total(values, i + 1)`.

---

**Part F — Write the code****W3-C1**

```
def array_sum(arr, i=0):
    if i >= len(arr):
        return 0
    return arr[i] + array_sum(arr, i + 1)  # nothing left
```

**W3-C2**

```
def count_char(text, ch):
    if text == "":
        return 0
    return (1 if text[0] == ch else 0) + count_char(text[1:], ch)
```

Correct, but  $O(n^2)$  because of the slice — pass an index to make it  $O(n)$ .

### W3-C3

```
def is_sorted_rec(arr, i=0):
    if i >= len(arr) - 1:
        return True
    if arr[i] > arr[i + 1]:
        return False
    return is_sorted_rec(arr, i + 1)
```

*# zero or one element left*

### W3-C4

```
def no_consecutive_ones(n):
    def build(prefix, remaining):
        if remaining == 0:
            return [prefix]
        result = build(prefix + "0", remaining - 1)
        if not prefix.endswith("1"):
            result = result + build(prefix + "1", remaining - 1)
        return result
    return build("", n)
```

Counts for  $n = 1, 2, 3, 4, 5$ : **2, 3, 5, 8, 13** — the Fibonacci numbers. A valid string of length  $n$  either starts with 0 followed by any valid string of length  $n - 1$ , or starts with 10 followed by any valid string of length  $n - 2$ , so  $\text{count}(n) = \text{count}(n - 1) + \text{count}(n - 2)$ .

### W3-C5

```
def pascal_row(n):
    if n == 0:
        return [1]
    previous = pascal_row(n - 1)
    middle = [previous[k - 1] + previous[k] for k in range(1, len(previous))]
    return [1] + middle + [1]
```

Each entry is the sum of the two above it. One recursive call per row: depth  $n$ , and  $O(n^2)$  additions in total.